
Next-Pitch Prediction in Baseball

Arvind Madan, Daniel Fulk, Elliott Walker

Data 780 - Machine Learning

University of North Carolina Chapel Hill - MADS

mxngo@ad.unc.edu, djf31702@ad.unc.edu, elliottw@unc.edu

<https://github.com/elliottw-23/pitch-forecaster>

Abstract

The goal of our project is to build a next-pitch model: “what type of pitch will a batter see coming from the mound next?” In this paper, we provide a project progress report. First, we answer the question of “why” baseball as the domain for our research and establish a narrative between the prevalence of data science in baseball, and where our motivations for this project lie. We then explore published research in this space with the goal of understanding where previous attempts found successes and failures. Finally, we take a deeper look at the data involved and finish with a tour of the project plan.

1 Introduction

That baseball is tightly intertwined with the practice of data science is hardly a novel concept. The inexorable prevalence of data and data analysis in the sport can be traced back generations. The Box Score was introduced in 1858. Branch Rickey of the Brooklyn Dodgers, an innovator of the sport in the 1950’s, is known for introducing statistical analysis into decision-making (in addition to the ‘farm system’ and batting cages). The founding of SABR (Society for American Baseball Research) by Bill James in the 1980’s led to the eponymous practice of “sabermetrics”, which now effectively denotes the inclusion of advanced analytics in all sports. Most recently, the marriage of data analysis and baseball hit the mainstream consciousness by way of Micheal Lewis’ book Moneyball, and the adjoining movie adaptation. These sold 1.7 million books and \$111 million at the box office, respectively.

A fair question, therefore, would be: “why choose baseball for this project, if it is indeed oversaturated with numerical analysis?” The answer is multifaceted.

Baseball analytics is an ongoing arms race. The most immediate reason is because baseball has hardly been “solved”. Per MLB’s Statcast: “Each MLB organization now has an analytics team using data to gain a competitive advantage. Clubs are unwilling to disclose exactly how they are using the data, engaging in an ‘arms race’ of data analysis. This ‘arms race’ of new data that is becoming available is a rapidly growing field within Major League Baseball teams.” In other words, baseball teams maintain strategic urgency towards the discovery and re-discovery of trends, philosophies, and in-game likelihoods.

Our project aims to predict the next pitch. This, as a capability in the hands of an MLB dugout, would be an immediate and substantive competitive advantage in the battle between teams’ analytics departments. While not in scope for this project, a natural next step could be to quantify the impact of knowing what pitch might come next versus the actual impact it had on the outcome of the play (or game).

Baseball is an approachable domain with good data. The game itself is largely unchanged in the ~160 years since its invention. Its simplicity means that it is easily approached by the layperson, and its nuance means that it lends itself well to being the statistician’s favorite subject. As a corollary:

because baseball has been around for so long, and because it is beloved by “numbers” people, baseball data is easy to come by. We explore data source selection in the next section, but it was not difficult to source clean, recent, and well-documented data. As importantly, because of this approachability, baseball makes a phenomenal sandbox for exploring machine learning concepts, seeing models interact with the data, and being able to intuit the results back to the game. We can (and will) explore several differing approaches, experiencing the gamut of model selection and evaluation.

Field-deployable model. As baseball is a good foil for data analysis, it is also timely: the season starts in the spring and ends in October. Thus, the culmination of this project is a model that we can immediately use in the wild when watching late-season baseball. In addition to the satisfaction of impressing friends and family when at a game, there is a substantive engineering (and learning) challenge to tackle. Learning how to put a model into practice – that is, deploying, versioning, and testing against organically new data – will be a great experience that can continue to pay dividends beyond the formal initial project delivery.

In the next sections, we catalog and detail some of the approaches that others have taken when thinking about the next-pitch problem. Understanding their successes and failures will be instrumental in building a foundation on which we can confidently move forwards. Finally, we conclude this mid-point report with a project timeline – including primary components, resources needed, and priority of our efforts.

Practical Application. While our model may initially seem like a form of sign-stealing, it’s important to note that we’re only using historical, publicly available data - not live signals or confidential information. That said, predictive models like this could have a variety of other applications. With the growing popularity of in-game betting, for instance, knowing the likely next pitch could inform smarter wagers. Broadcast networks could also benefit - Apple TV already displays real-time stats such as a batter’s likelihood of reaching base. A model like ours could extend those insights by adding probabilities for the next pitch type, enriching both fan engagement and commentary.

2 Related Work

As discussed, baseball is a well-populated part of the data science world. “Researchers often use statistical analysis to question traditional measures of baseball evaluation such as batting average and pitcher wins,” says the splash page to the SABR website. We had no preclusions, therefore, that similar research wouldn’t have already been conducted. This also meant that we did not have to look far to find similar approaches from which to draw inspiration and ideas. From the (significant) amount of research conducted on the “art” of pitching, our first task was to winnow out the research related to our exact topic. (Anecdotally, some interesting-but-not-quite-useful research was: the use of wearables in pitch type classification; the impact of certain physical characteristics on pitcher efficacy; bullpen ordering and optimization.) Among all of the papers we read, four stood out as having made material strides towards accomplishing the end goal. Moreover, these four best represent how the problem can be approached from various different directions, as well as give us a decent sense of relative performance and complexity of each. Below, we explore these four related works.

Support Vector Classification via amateur github link. The first model to examine is the simplest in terms of its scope and scale. It was posted on the r/Sabermetrics reddit (we can see the project in its entirety on github) and while this amateur approach does not constitute serious research, it does provide a decent eye-test for how one might look at this problem. This author chose to implement a pitch-type classification system using a Support Vector Classification (SVC) model. While their approach encodes characteristics of the pitch (speed, break, pitcher), it fails to use any situational data such as innings, outs, handedness, etc. The model itself stops short of trying to predict any next pitch, and remains focused on the previous/current pitch. Ultimately, for this reason, we can see that the SVC model is probably too limited on its own.

Two complete Random Forest approaches. We get two full end-to-end implementations of our next-pitch prediction idea courtesy of the Medium blog site. The first, which we will call the “Fischer” model, provides a straightforward path for implementing a random forest algorithm. A large portion of the writeup for the Fischer model talks about the importance of feature selection; they go with pitch type, location, velocity, batter and pitcher handedness, and game situation. The inclu-

sion of game situations is a clear step in the right direction for us – as we consider these features important when deciding next in a series.

The “Augustine” model concurs with the other in terms of importance of game-situational data. It implements a Multiclass Decision Forest model, a similar ensemble method to the Random Forest, but where the trees are tasked with voting on a class, rather than the random bagging found in the RF model. One important departure was in the amount of data seen: Augustine focused on a single pitcher, which Fischer’s model saw a significant amount more data.

The primary takeaways here are: data is important, and having features relevant to the “next pitch” is important; and both models reported accuracy in the 65-75% range, which seems to be quite reasonably effective for the next-pitch task.

Directed Graph Sequencing model. The final model for review is a formally published paper from MIT Sloan’s Sports Analytics Conference. This paper introduces a novel approach to “pitch sequencing” by representing sequences as directed graphs and embedding them into a fixed-dimensional Euclidean space using the Sequence Graph Transform (SGT). Rather than focusing solely on individual pitch types, the method captures both short- and long-term dependencies within pitch sequences. Importantly, it intends to model strategic intent, such as “setup” and “knockout” pitches. These graph embeddings are then clustered using unsupervised learning to uncover universal sequencing patterns across pitchers and game contexts, which presents a singular, generalizable model.

This impressive model serves as the top-end, cutting edge of our aforementioned “arms race”.

Contrast with our approach. For this project, we drew inspiration from -and implemented- the Random Forest approach before also trying a Neural Network and then ultimately an Ensemble model combining both. We (correctly) guessed that the DAG Sequence would be ambitious, and after some deliberation decided to leave it for future experimentation. Moreover, we figured that the Neural Network would likely yield better results than the more simplistic SVC model.

3 Methodology

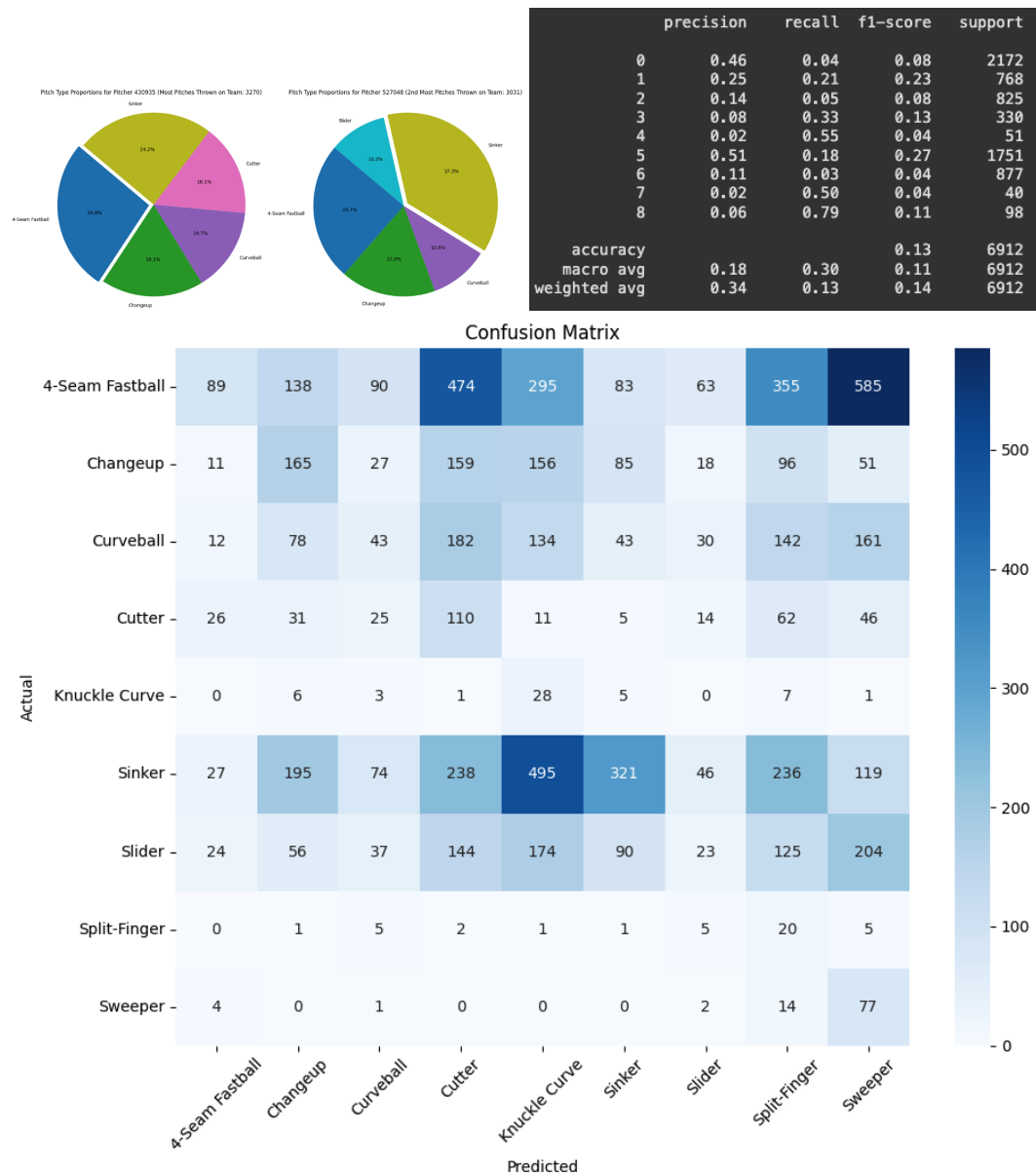
After loading pitch data for every play of every game in the Texas Rangers 2016 season, the first thing we needed to accomplish was cleaning/preparing the data to fit a logistic regression model for multi-class classification. The goal of this model is to take data that provides context regarding the game at a specific point then predict the type of pitch that will be thrown next. After removing the unnecessary variables that were included in the dataset along with pitches types that were not thrown enough to be represented in the model (Slow Curve, Pitch Out, and Intentional Ball), we had to create lagged versions of some variables to capture information about the play before the one we’re predicting for. This was done while grouping by individual games and pitchers, this ensures that the first play of one game won’t reference the last play of the prior game and allows the lagged variables to account for the last play each specific pitcher was on the mound.

An unfortunate by-product utilizing lagged variables, even when only lagging by one play, is that this introduces NA entries for plays where there is no prior play to reference. This forced us to exclude 656 records from the dataset that include a lagged NA value, but we were still left with 23,099 records to train/test this initial logistic model. We also engineered new features to decrease the total number of features in the model and combat collinearity in the features. The variables describing the handedness of the pitcher and batter were combined into a single binary variable that is 1 when the pitcher and batter have the same handedness and 0 if they have different handedness. The pitching team scores and batting team scores were also combined to make a single score differential variable for each play.

Once the data had been cleaned and all the features were engineered, we put all of the numeric variables into a correlation matrix to ensure that none of the features were overly correlated. After this was confirmed we fit a multinomial logistic regression to the training data (70% of the entire dataset) using scikit-learn with the lbfgs solver and balanced class weights. The model has nine possible pitch types that it can classify: 4-Seam Fastball, Changeup, Curveball, Cutter, Knuckle Curve, Sinkers, Slider, Split-Finger, and Sweeper.

4 Exploratory Data Analysis Results

Our first set of preliminary results comes from the EDA phase where we visualized the proportion of each pitch type thrown for the two pitchers on this team with the most total pitches thrown. We see that the main pitcher for this team primarily relies on the Sinker and 4-Seam Fastball equally at 24.2% and 26.8% respectively. While he throws the Changeup, Curveball, and Cutter less frequently at 18.1%, 14.7%, and 16.1% respectively. The pitcher with the second-most pitches thrown uses the 4-Seam Fastball at a similar rate to the main pitcher (24.7%), but relies more heavily on a single pitch—throwing a Sinker for 37.3% of his pitches. The second pitcher also defers outside of his main two pitches far less than the primary pitcher. This tracks with a general trend in the MLB where relief pitchers tend to rely more heavily on a few dominant pitches rather than throwing many pitch types.



Our second set of preliminary results comes from the logistic classification model we created. Given that we are attempting to capture rather complex relationships in the data we expected that this model would have relatively poor performance, and it did. For the 9 pitch types this model could predict, the lowest f1-score was a value of 0.04 and coincided with the Knuckle Curve, Slider, and Split-Finger

pitch types. The highest f-1 score of the pitch types was a value of .27 for the Sinker pitch type. The overall accuracy of this model was .13 on our testing data which consisted of 6,912 records. Of course this initial model is far from ground breaking, but this will serve as a good reference point to see how much we're really improving from the more classical approach as we attempt more complex methods for this classification problem. The full classification report and confusion matrix are shown above. The numeric encodings of the pitch types in the report correspond to the order of the pitch types along the predicted axis of the confusion matrix.

5 Methods

Our system integrates data from three key APIs: statcast, statcastpitcher, and baseballsavant, with baseballsavant serving as the foundational source that supports both statcast APIs through bulk-access to historical pitch-level data.

To support model training on large datasets, we designed a pipeline that streams data directly from these sources, enabling on-the-fly transformation and ingestion without the need to load the entire dataset into memory.

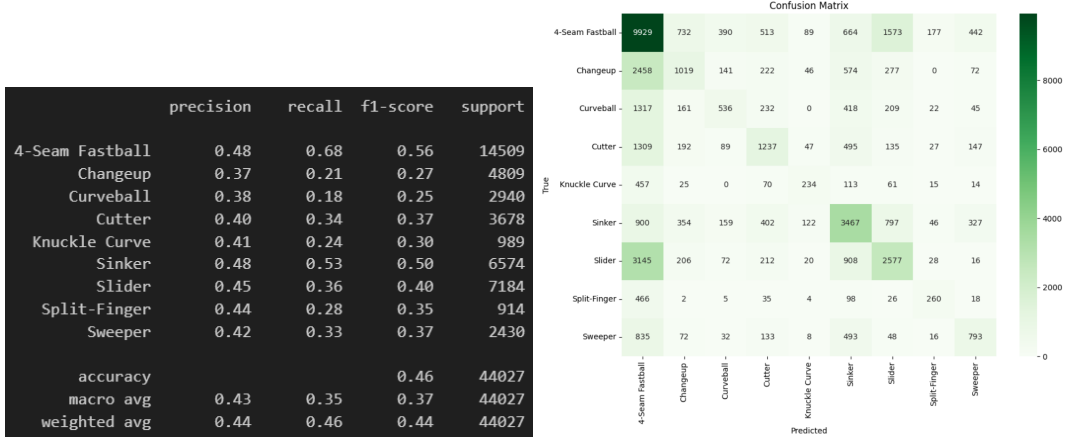
Several important transformations are applied during this process. These include the creation of lag features that encode information about previous pitches within a sequence, enrichment with pitcher arsenal characteristics that describe a pitcher's overall tendencies, enrichment with batter arsenal characteristics, and comprehensive data cleaning procedures that address outliers and inconsistencies in pitch type labels. To enable the arsenal enrichments - both for pitcher and batter, we prepopulate caches called `PitcherArsenalCache` and `BatterArsenalCache` using bulk exports from baseballsavant, while also supporting ad-hoc lookups using statcastpitcher for more targeted enrichment. All data transformation occurs within a Python generator-based pipeline, which serves as the backbone for streaming. This design allows us to produce small static test batches or large multi-season datasets using the same underlying infrastructure. To interface with machine learning models built in TensorFlow/Keras, we implement a custom class that extends `keras.utils.Sequence`. This class supports memory-efficient model training by feeding data using `train-on-batch()`, and it also manages dynamic splitting into training, validation, and test sets. The result is a flexible, scalable pipeline capable of producing clean, well-structured X and y data ready for use in any Keras-compatible model.

6 Experiments/Results

The logistic regression model (built during the EDA phase) established a baseline with moderate accuracy of 40 percent and modest F1 scores across pitch types. It performed best on frequent pitches like four-seam fastballs ($f1 = 0.42$) and sliders ($f1 = 0.45$), but struggled with rarer pitches such as knuckle curves ($f1 = 0.31$). The confusion matrix shows that many errors involved misclassifying rare pitches as their more common counterparts - for example, fastballs and sliders had high rates of false positives. This provides a useful reference point for evaluating the gains from more flexible, nonlinear models.

6.1 Random Forest

Building on this, we trained a random forest classifier using the same data preparation and class-weighting scheme. The model was trained with 100 trees and a fixed random seed for reproducibility. Random forests work well in this context because they can model more complex, nonlinear relationships between features. We assessed the performance of the random forest model using a classification report and confusion matrix based on its predictions on the test set.



The random forest model improved over logistic regression with an overall accuracy of 46%. It performed well for dominant pitch types like four-seam fastballs ($f1 = 0.56$) and sinkers ($f1 = 0.50$). However, performance declined on less frequent pitches such as curveballs ($f1 = 0.25$), splitters ($f1 = 0.35$), and knuckle-curves ($f1 = 0.30$). Even with class weighting, the model leaned toward predicting majority classes, as seen in the confusion matrix. Its macro F1 score of 0.37 highlighted this class imbalance, but the model still offered a solid foundation for nonlinear modeling.

6.2 Neural Network

To enhance model performance beyond our baseline, we implemented a neural network and tuned its architecture using cross-validation.

Cross-Validation and Hyperparameter Tuning Instead of using a traditional random k-fold cross-validation, we opted for a more meaningful split based on MLB divisions. We grouped teams into three buckets - East, Central, and West - regardless of whether they belonged to the American or National League. For each fold, we trained on two divisions and validated on the third, which allowed us to evaluate how well the model generalized to entirely new teams and game contexts.

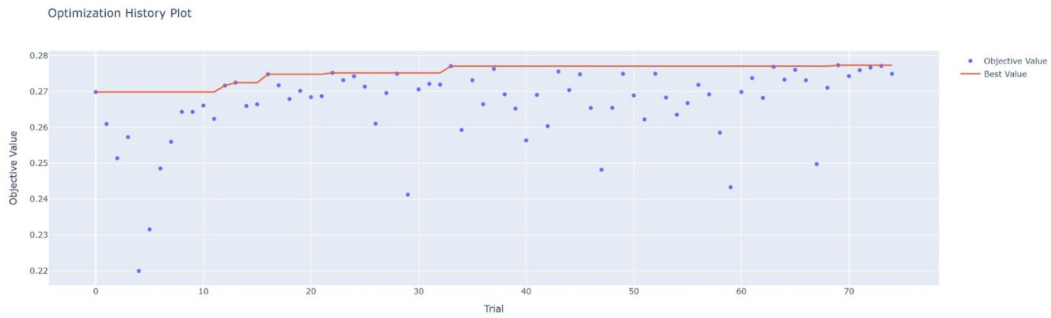
To tune our neural network effectively, we used Optuna, a powerful optimization framework that helps find the best combination of hyperparameters. Because neural networks have so many tunable parameters, it's practically impossible to try every possible combination manually. Optuna handles this challenge by running multiple trials and using smarter search strategies based on past results to focus on the most promising options. This allowed us to efficiently explore a large range of configurations and identify the ones that gave us the best performance across our cross-validation folds.

The hyperparameters we tested included architectural parameters like the number of hidden layers, the number of neurons per layer, and whether to use a third hidden layer at all. We tested both ReLU and Leaky ReLU activations, tuning the leaky alpha when applicable, and experimented with various dropout rates to prevent overfitting. We also varied training-related hyperparameters such as learning rate, batch size, number of epochs, early stopping patience, and, again, the custom scaling factor for class weighting. We evaluated each fold using a hybrid metric to balance overall accuracy and per-class fairness:

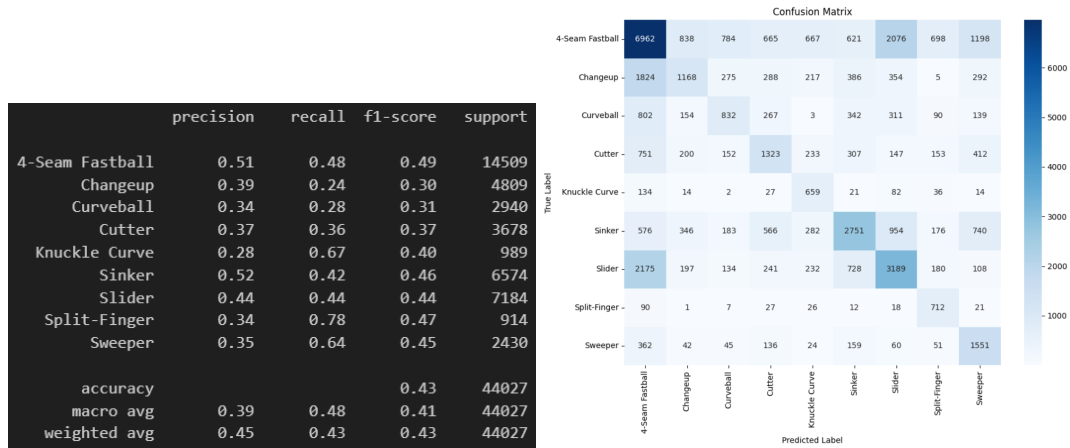
$$HybridScore = 0.2Accuracy + 0.5Mean_{F1} + 0.3Minimum_{F1}$$

After running 75 trials, Optuna identified a neural network configuration that achieved a hybrid score of 0.2921. The best-performing model used two hidden layers with 256 and 64 neurons respectively, Leaky ReLU activation ($\alpha = 0.005$), and applied 50% dropout after the first layer. It was trained with an L2 regularization strength of 0.001, a learning rate of 0.005, and a relatively low class weight scaling factor ($\alpha = 0.2$). The model trained for up to 30 epochs with early stopping ($patience = 5$), and used a batch size of 4096. Notably, the third layer was disabled in this configuration.

The following plot shows how the model’s performance improved over the course of the hyperparameter search. Each point is a trial with its hybrid score. You can see that Optuna gradually found better models as it learned from past results. The best trial came on the 71st attempt.



To evaluate the final neural network model, we report its classification metrics and confusion matrix on a held-out validation set.

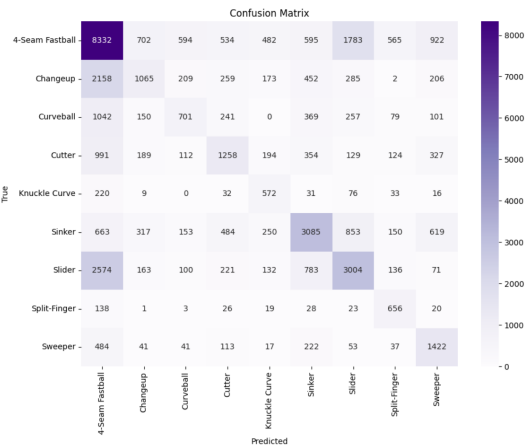


The neural network handled underrepresented pitch types more effectively than earlier models. Notably, it achieved strong recall for knuckle-curves ($recall = 0.67$) and split-finger fastballs ($recall = 0.78$), while maintaining decent performance on common pitches like four-seam fastballs ($f1 = 0.49$) and sliders ($f1 = 0.44$). The macro F1 score of 0.41 reflected a more even distribution of predictions across classes. However, overall accuracy decreased to 43%.

6.3 Ensemble Models

To enhance predictive performance, we implemented an ensemble model that linearly combines the outputs of our neural network and random forest classifiers. The motivation for ensembling was to leverage the strengths of both models: the neural network, with its ability to model complex, nonlinear patterns, performed better on class-wise F1 scores, while the random forest yielded higher overall accuracy and interpretability. By combining them, we aimed to balance general predictive strength with fairness across pitch classes. We experimented with various weighted combinations of the two models and ultimately selected a 70/30 split (assigning 70% weight to the neural network and 30% to the random forest). This configuration offered the best trade-off between mean F1 score and accuracy, improving overall robustness while mitigating class imbalances. The resulting ensemble was evaluated on the same validation set as the individual models, and its performance is summarized below using both a classification report and confusion matrix. This allows us to directly compare its accuracy and class-wise metrics against the standalone neural network and random forest models.

	precision	recall	f1-score	support
4-Seam Fastball	0.50	0.57	0.54	14509
Changeup	0.40	0.22	0.29	4809
Curveball	0.37	0.24	0.29	2940
Cutter	0.40	0.34	0.37	3678
Knuckle Curve	0.31	0.58	0.40	989
Sinker	0.52	0.47	0.49	6574
Slider	0.46	0.42	0.44	7184
Split-Finger	0.37	0.72	0.49	914
Sweeper	0.38	0.59	0.46	2430
accuracy			0.46	44027
macro avg	0.41	0.46	0.42	44027
weighted avg	0.46	0.46	0.45	44027



The ensemble model produced the most balanced results across all metrics. It improved minority class F1 scores - such as 0.40 for knuckle-curves, 0.49 for splitters, and 0.46 for sweepers - without sacrificing performance on common pitches like four-seam fastballs ($f1 = 0.54$) or sinkers ($f1 = 0.49$). The macro F1 score increased to 0.42, and accuracy remained strong at 46%. By combining the strengths of both models, the ensemble made fewer extreme mistakes and produced more balanced predictions across pitch types.

7 Conclusion

In this project, we set out to answer a deceptively simple question: “What pitch is coming next?” Our work throughout this project produced a functional and flexible machine learning pipeline capable of consuming pitch-level data from Sabermetric-favored APIs, transforming it into model-ready inputs, and training classifiers to predict the next pitch type thrown. We implemented and evaluated multiple approaches: we started with a logistic regression baseline, progressed through random forest and neural network models, and ended with an ensemble model that prioritized accuracy and stability across class-uneven pitch types. Our best ensemble achieved a macro score of ($F1 = 0.42$) and an overall accuracy of 46%, demonstrating meaningful predictive power in a complex, highly imbalanced domain.

Through this process, we learned that pitch prediction is not just a matter of modeling individual pitch characteristics, but also one where understanding context is of the utmost importance. Understanding competitive context, pitch sequencing, handedness matchup, the game state, and pitcher-batter tendencies all contribute meaningfully. We also discovered that traditional accuracy metrics often obscure poor performance on underrepresented pitch classes, making it essential to evaluate models with metrics that account for class imbalance. Our approach contributes to the broader field of sports analytics in several ways. First, by designing a scalable, streaming-based data pipeline, we lay the groundwork for real-time and historical model training that can adapt to new seasons or expanded datasets. Second, our cross-validation strategy, based on MLB divisions, demonstrates a novel method for measuring model generalization across teams. Finally, our hybrid metric for model selection, which balances accuracy, mean F1, and minimum F1, provides a more holistic view of performance in multiclass imbalanced settings like this one.

Looking Ahead There are several promising ways to continue improving the model. One major opportunity is incorporating batter-specific hit tendencies by pitch type (i.e., how often a given batter gets hits on fastballs versus sliders). This would provide useful context for matchup-specific outcomes, such as how a pitcher might change their pitch selection based on a batter’s ability to handle certain pitch types. Another potential enhancement is applying dimensionality reduction techniques like autoencoders or PCA to compress high-dimensional input features while preserving meaningful structure, which could help regularize the model and improve generalization. Finally, training on more data, such as multiple seasons, could significantly boost performance. We were limited by hardware constraints, particularly RAM, during this iteration. Addressing these limitations in future work could unlock additional improvements in predictive power and robustness. Our progress thus far confirms that while pitch prediction is a difficult task, but not unsolvable, and that a thoughtful, engineering-aware approach to modeling can yield insights both statistically and practically mean-

ingful in the modern game. At the same time, the imperfections in our models serve as a reminder of what makes baseball so captivating. The fact that we cannot perfectly predict the next pitch preserves the human element; the spontaneity, strategy, and split-second decisions make the sport unpredictable and enjoyable.

References

- [1] dbc. *pybaseball*. <https://github.com/jldbc/pybaseball>.
- [2] Major League Baseball. *Statcast Search – Baseball Savant*. https://baseballsavant.mlb.com/statcast_search.
- [3] Stats Perform. *State of Analytics: How the Movement Has Forever Changed Baseball – For Better or Worse*. <https://www.statsperform.com/resource/state-of-analytics-how-the-movement-has-forever-changed-baseball-for-better-or-worse/>.
- [4] Smithsonian Institution. “Branch Rickey: Baseball Innovator.” *National Museum of American History*. <https://invention.si.edu/invention-stories/branch-rickey-baseball-innovator>.
- [5] Wikipedia contributors. “Henry Chadwick (Writer).” *Wikipedia, The Free Encyclopedia*. [https://en.wikipedia.org/wiki/Henry_Chadwick_\(writer\)](https://en.wikipedia.org/wiki/Henry_Chadwick_(writer)).
- [6] Society for American Baseball Research. “Sabermetrics.” *SABR*. <https://sabr.org/sabermetrics>.
- [7] Shaikin, Bill. “Statcast Era: How Data and Technology Are Changing Baseball.” *Sports Illustrated*, 26 Aug. 2016. <https://www.si.com/mlb/2016/08/26/statcast-era-data-technology-statistics>.
- [8] Wikipedia contributors. “Statcast.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/Statcast>.
- [9] Toddrob99. “MLB-StatsAPI.” *GitHub*. <https://github.com/toddrob99/MLB-StatsAPI>.
- [10] Wikipedia contributors. “PITCHf/x.” *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/wiki/PITCHf/x>.
- [11] Kim, Joon-Ho, et al. “Machine Learning Approach for Pitch Type Classification Based on Pelvis and Trunk Kinematics Captured with Wearable Sensors.” *Sensors*, vol. 23, no. 23, 2023, p. 9373. <https://www.mdpi.com/1424-8220/23/23/9373>.
- [12] Bergeron, Kara. “Predicting Baseball Pitcher Efficacy Using Physical Pitch Characteristics.” *Emerging Investigators*, 2022. <https://emerginginvestigators.org/articles/22-270>.
- [13] Prasad, Arnav. “Decoding MLB Pitch Sequencing Strategies via Directed Graph Embeddings.” 2021. https://cdn.prod.website-files.com/5f1af76ed86d6771ad48324b/606e515d0e2589d01b6946ba_ArnavPrasad-DecodingMLB-RPpaper.pdf.
- [14] Prasad, Arnav. “Decoding MLB Pitch Sequencing Strategies via Directed Graph Embeddings.” *MIT Sloan Sports Analytics Conference*. <https://www.sloansportsconference.com/research-papers/decoding-mlb-pitch-sequencing-strategies-via-directed-graph-embeddings>.
- [15] Fisher, Michael. “Decoding the Next Pitch: Using Data to Anticipate a Pitcher’s Move in Baseball.” *Medium*, 2023. <https://mafisher.medium.com/decoding-the-next-pitch-using-data-to-anticipate-a-pitchers-move-in-baseball-d897bfea4dd2>.
- [16] Jakeenea51. “MLB Pitch Identification with ML.” *GitHub*. <https://github.com/jakeenea51/MLB-Pitch-Identification-with-ML>.
- [17] NeurIPS. “NeurIPS 2024 Papers.” *NeurIPS*, 2024. <https://nips.cc/virtual/2024/papers.html>.

- [18] NeurIPS. “Poster 96215.” *NeurIPS 2024 Virtual Conference*. <https://nips.cc/virtual/2024/poster/96215>.
- [19] NeurIPS. “NeurIPS 2018 Style Files.” *NeurIPS*. <https://nips.cc/Conferences/2018/PaperInformation/StyleFiles>.